Odoo 19 Enterprise — Demo Study

Cost, Inventory, Procurement & Supply Chain, General Ledger (Accruals, VAT, Banks, Assets) and Manufacturing

---

Source-verified against the local Odoo 19 codebase

3 August 2026

Introduction

This document is a demo-preparation study of **Odoo 19** across eight business domains:

1. Cost Management
2. Inventory Management
3. Procurement & Supply Chain
4. General Ledger (with accrual/deferral schemes — rent, electricity)
5. VAT / Taxes
6. Banks
7. Fixed Assets
8. Manufacturing

For each domain it answers three questions: **Is it covered in Odoo 19 Enterprise?**, **Which modules cover it (and are they Community or Enterprise)?**, and **What is the end-to-end business cycle in detail?** — plus a click-through demo script.

Everything below was **read from the actual Odoo 19 source** installed locally (Community at `/odoo/odoo-19-community/addons`, Enterprise at `/odoo/odoo-19-enterprise`), not from memory. Model names, field names, states, and file paths are cited so the demo narrative is accurate to this exact version.

> **Reading guide.** "Community" means the feature ships in the free/LGPL base and is therefore also present in Enterprise. "Enterprise" means it ships only in the paid edition. Odoo Enterprise = Community + the Enterprise modules, so an Enterprise customer has **everything** below.

Headline coverage answer

Yes — all eight domains are covered in Odoo 19 Enterprise. The split is:

Domain	Covered in EE?	Core engine edition	Enterprise-only pieces
Cost Management	Yes	Community (`stock_account`, `stock_landed_costs`)	Inventory Valuation audit report, advanced reporting
Inventory Management	Yes	Community (`stock`)	Barcode app, cohort/map reporting
Procurement & Supply Chain	Yes	Community (`purchase`, `stock`, `purchase_requisition`)	3-way match, Master Production Schedule
General Ledger	Yes	Community (`account`)	**Deferred expense/revenue automation**, all managerial financial reports
VAT / Taxes	Yes	Community data model (`account`)	**Tax Report + VAT return / tax closing**, cash-basis reports
Banks	Yes	Community data model (`account`)	**Bank reconciliation widget**, online sync, SEPA/ISO 20022, statement import, batch payments
Fixed Assets	Yes	**Enterprise only** (`account_asset`)	Entire module (auto-installs with Accounting)
Manufacturing	Yes	Community (`mrp`)	Shop Floor, PLM/ECO, MPS, Quality, cost-analysis report

Two version-specific corrections to carry into the deck (they trip up anyone briefing from Odoo 16–18 memory):

- **Inventory valuation was rewritten in v19.** The `stock.valuation.layer` (SVL) model **no longer exists**. Valuation now lives directly on `stock.move.value`, manual revaluations are journaled in a new `product.value` model, and the two valuation modes were relabelled **"Periodic (at closing)"** and **"Perpetual (at invoicing)"**.

- **The VAT closing was refactored** from a company-cron into a first-class `account.return` object (`account_reports`). And **Work Orders** (`mrp.workorder` + start/pause/finish) actually ship in **Community** `mrp` ; Enterprise `mrp_workorder` adds the Shop Floor UX, not the model.

1 & 2. Inventory Management & Cost Management

> **Version note.** The Odoo 19 stock-valuation engine was rewritten. `stock.valuation.layer` (used in 14–18) is **gone**; valuation is stored on `stock.move.value`, manual re-valuations/price changes are journaled in a new `product.value` model, and the valuation mode is now **"Periodic (at closing)"** / **"Perpetual (at invoicing)"**. A demo script inherited from older versions will be wrong on the data model.

1.1 Modules, paths, edition

Technical name	Purpose	Path	Edition
`stock`	Core inventory: warehouses, locations, operation types, transfers, moves, quants, lots/serials, packages, routes, rules, reordering	`/odoo/odoo-19-community/addons/stock`	**Community**
`product`	Products, variants, categories, UoM, and the **costing-method** selection (`cost_method`) on the category	`/odoo/odoo-19-community/addons/product`	**Community**
`stock_account`	Valuation + accounting engine: move valuation, cost methods, stock/variation accounts, periodic-closing entries, manual revaluation	`/odoo/odoo-19-community/addons/stock_account`	**Community**
`stock_landed_costs`	Landed costs: split extra costs (freight, duty, insurance) across received goods and adjust valuation	`/odoo/odoo-19-community/addons/stock_landed_costs`	**Community**
`analytic`	Analytic objects used for cost tracking; stock moves post analytic lines	`/odoo/odoo-19-community/addons/analytic`	**Community**
`stock_enterprise`	Advanced Stock **reporting** — cohort + map views, pivots; auto-installs with stock	`/odoo/odoo-19-enterprise/stock_enterprise`	**Enterprise**
`stock_accountant`	Bridge Stock ↔ Accounting: **Inventory Valuation** audit report; clean reconciliation-widget behaviour; auto-installs	`/odoo/odoo-19-enterprise/stock_accountant`	**Enterprise**
`stock_barcode`	Barcode-driven warehouse operations on desktop/mobile scanners	`/odoo/odoo-19-enterprise/stock_barcode`	**Enterprise**

The operational and costing core is Community; Enterprise layers barcode operations, the valuation audit report, and advanced analytics on top without replacing anything.

1.2 Inventory operations cycle

Everything physical is a **`stock.move`** (demand to move a quantity between two locations), executed through **`stock.move.line`** records and grouped into a **`stock.picking`** (a transfer) of an **operation type** (`stock.picking.type` : Receipts, Internal Transfers, Delivery Orders, Returns, Manufacturing…).

Picking lifecycle (`stock.picking.state`): `draft` → `waiting` → `confirmed` (awaiting availability) → `assigned` (Ready, stock reserved) → `done` (validated) — plus `cancel` . Validating (`button_validate` → `_action_done`) is when stock actually moves: quants update and, for valued moves, valuation and accounting are written.

Locations (`stock.location`) are a typed tree: `internal` , `supplier` / `customer` , `transit` , `inventory` (adjustment counterpart), `production` , `view` . Valuation hinges on type — a move **into** a valued

(internal/transit) location is *incoming*; the reverse is *outgoing*.

The operational cycle:

1. **Receipts** (Vendor → Input/Stock) per the warehouse's `reception_steps`: `one_step` (Receive), `two_steps` (Input → Stock), `three_steps` (Input → QC → Stock).
2. **Internal transfers** (Stock ↔ Stock, multi-step hops, inter-warehouse resupply).
3. **Deliveries** (Stock → Customer) per `delivery_steps`: `ship_only`, `pick_ship`, `pick_pack_ship`. Intermediate Input/QC/Pack/Output locations are auto-created per warehouse.
4. **Routes & rules** (`stock.route` + `stock.rule`): ordered push/pull rules deciding *how* demand is met (MTO is a pull chain).
5. **Reordering rules** (`stock.warehouse.orderpoint`): min/max triggers surfaced in the **Replenishment** view.
6. **Lots & serial numbers** (`stock.lot`): `tracking` = none/lot/serial, full traceability; in v19 a lot can carry its own valuation.
7. **Packages** (`stock.package` / `stock.package_type` / `stock.storage.category`): put-away by storage category.
8. **Inventory adjustments** on **`stock.quant`** (product × location × lot × owner → qty); applying a count generates an adjustment move, and in v19 the quant exposes an **Accounting Date** so valuation books to a chosen period.

1.3 Cost management cycle

Costing method (`product.category.property_cost_method`, default from `res.company.cost_method`):

Method	Behaviour
`standard`	Every unit valued at `standard_price`; purchase-price differences go to a price-difference account.
`average`	AVCO — weighted moving average; `standard_price` recomputed on each incoming move.
`fifo`	Outgoing moves consume the oldest incoming layers; value walked over the stack of incoming moves.

Valuation mode (`product.category.property_valuation`, default from `res.company.inventory_valuation`):

- **`periodic` — "Periodic (at closing)"**: no per-move entry; a **closing entry** reconciles theoretical inventory value to the accounting balance, run manually or on a schedule.
- **`real_time` — "Perpetual (at invoicing)"**: an `account.move` is posted **per valued move** at validation.

Where value lives now (v19 refactor):

- `stock.move.value` — the monetary value of the move, computed in `_action_done`.
- `product.value` — audit trail of manual value changes (standard-price edits, the "Adjust Valuation" wizard).
- `stock.quant.value` — computed on-hand value for reporting.
- `product.product.total_value` / `avg_cost` — computed live from moves (no more summed SVL rows).
- Lot-level valuation — with `lot_valuated`, each `stock.lot` carries its own value.

Accounting entries (perpetual). For a valued move, `_create_account_move` posts to the **Stock Journal**:

- **Incoming**: Dr Stock Valuation / Cr the source location's input counterpart (Stock Input / GRNI).
- **Outgoing**: Dr the destination's output counterpart (Stock Output / COGS) / Cr Stock Valuation.

Anglo-Saxon vs Continental behaviour is driven by `product.category.anglo_saxon_accounting`.

Accounting entries (periodic / closing). `res.company.action_close_stock_valuation()` (also run by a cron per `inventory_period`) compares **theoretical** inventory value against the **accounting balance** and books the difference to the **Stock Variation** account. Closes are tracked so you cannot back-date before the last close.

Landed costs cycle (`stock.landed.cost`, states `draft` → Posted → `cancel`):

1. **Create** a landed cost, choose target = Transfers, select the receipts; optionally link the vendor bill carrying the freight/duty.
2. **Add cost lines** — each a service product with an amount and a **split method**: `equal`, `by_quantity`, `by_weight`, `by_volume`, or `by_current_cost_price`.
3. **Compute** generates **valuation adjustment lines** (one per targeted move) allocating each cost by the chosen split. Only FIFO/AVCO moves are eligible (standard-price rejected).
4. **Validate** posts Dr Stock Valuation / Cr the cost's expense account and folds the extra cost into each move's value — the on-hand cost of the received goods rises by the allocated landed cost.

Revaluation. Editing a product's/lot's `standard_price` or the per-move **Adjust Valuation** wizard records a `product.value` and (for AVCO/FIFO) updates the live average. Negative landed costs reverse a posted landed cost.

Key models: `stock.picking`, `stock.move`, `stock.move.line`, `stock.quant`, `stock.lot`, `stock.location`, `stock.picking.type`, `stock.route`, `stock.rule`, `stock.warehouse.orderpoint`, `stock.warehouse`; costing side — `product.category`, `stock.move.value`, **`product.value`** (new), `stock.landed.cost`, `stock.landed.cost.lines`, `stock.valuation.adjustment.lines`. (No `stock.valuation.layer` in v19.)

1.4 Demo talking points

Inventory: flip a warehouse to 3-step incoming / Pick-Pack-Ship outgoing and show auto-created locations; validate a receipt through `draft → assigned → done`; enable lots and open the Traceability Report; adjust a quant with an Accounting Date; set reordering rules and run Replenishment; *(EE)* process a receipt in the Barcode app and open the cohort/map reporting views.

Cost: on a product category set Costing Method + Valuation mode; receive an AVCO/FIFO product and open the auto-posted journal entry (Dr Valuation / Cr Input), then deliver and show COGS; run a **Landed Cost** with a By-Weight freight split → Compute → Validate → show the product cost rise; change a cost via **Adjust Valuation** and show the `product.value` history; for a Periodic category run **Close Stock Valuation** and show the single Stock Variation entry; *(EE)* open **Accounting → Reporting → Audit → Inventory Valuation**.

3. Procurement & Supply Chain

3.1 Modules, paths, edition

Technical name	Path	Edition	Role
`purchase`	`/odoo/odoo-19-community/addons/purchase`	**Community**	RFQ/PO lifecycle, vendor bills, bill-control policy, PO alternatives base
`purchase_stock`	`/odoo/odoo-19-community/addons/purchase_stock`	**Community**	Bridge to inventory: Buy route/rule, receipts, `_run_buy`, replenishment, vendor lead time
`purchase_requisition`	`/odoo/odoo-19-community/addons/purchase_requisition`	**Community**	Purchase agreements: blanket orders + templates; PO alternatives (calls for tender)
`purchase_mrp`	`/odoo/odoo-19-community/addons/purchase_mrp`	**Community**	Links POs to manufacturing (kit/BoM cost, MO traceability)
`stock`	`/odoo/odoo-19-community/addons/stock`	**Community**	Procurement engine: `stock.rule`, `procurement.group`, `stock.route`, orderpoints, scheduler
`stock_dropshipping`	`/odoo/odoo-19-community/addons/stock_dropshipping`	**Community**	Dropship route + operation type (vendor → customer direct)
`sale_purchase`	`/odoo/odoo-19-community/addons/sale_purchase`	**Community**	Service/MTO for services (SO line → PO line)
`account_3way_match`	`/odoo/odoo-19-enterprise/account_3way_match`	**Enterprise**	"Release to Pay" 3-way match (PO vs receipt vs bill)
`mrp_mps`	`/odoo/odoo-19-enterprise/mrp_mps`	**Enterprise**	Master Production Schedule — forecast-driven replenishment
`purchase_intrastat`	`/odoo/odoo-19-enterprise/purchase_intrastat`	**Enterprise**	Intrastat declarations on vendor bills

The core procurement engine (routes, rules, orderpoints, scheduler) is entirely **Community** `stock`. Enterprise adds the financial control gate (3-way match) and forecast planning (MPS).

3.2 Procure-to-Pay cycle

Model `purchase.order`. State machine:

State	UI label	Set by
`draft`	**RFQ**	on creation
`sent`	**RFQ Sent**	`action_rfq_send()`
`to approve`	**To Approve**	`button_confirm()` over the double-validation threshold
`purchase`	**Purchase Order**	`button_approve()` / confirm
`cancel`	**Cancelled**	`button_cancel()`

1. **RFQ (draft)** — `purchase.order` + `purchase.order.line`; vendor, Expected Arrival (`date_planned`). No stock/accounting effect.
2. **Send** — mail composer → `sent`; portal lets the vendor view/acknowledge.
3. **Confirm** — validates lines, writes back a `product.supplierinfo` price/lead-time, then approves (→ `purchase`) or routes `to approve` if over the company threshold; sets `date_approve`, optionally locks the

PO.

4. **Receipt** — confirmation generates the incoming `stock.picking` / `stock.move`; validating sets `qty_received`.
5. **Vendor bill** — `action_create_invoice()` creates a bill (`in_invoice`) linked via `purchase_line_id`; `invoice_status` and `qty_to_invoice` drive billing. **Bill Control policy** (`product.purchase_method`): *On ordered quantities* vs *On received quantities* — the latter makes 3-way matching meaningful.
6. **3-way match (Enterprise)** — `account_3way_match` adds `release_to_pay` (`yes` / `no` / `exception`): `yes` = ordered & received & agree → payable; `no` = nothing received; `exception` = received/invoiced qty or price differ → blocked. `force_release_to_pay` overrides.
7. **Payment** — `account.payment` once the bill is posted (and, in EE, released to pay).

3.3 Replenishment engine

Routes & rules — a `stock.route` is an ordered set of `stock.rule` records; each rule's `action` is `pull` / `push` / `pull_push` and `procure_method` is `make_to_stock`, `make_to_order` (MTO), or `mts_else_mto`. The **Buy route** carries a Buy rule (`action='buy'`), created per warehouse and toggled by `buy_to_resupply`.

Reordering rules — `stock.warehouse.orderpoint` (Minimum Inventory Rule): `product_min_qty` / `product_max_qty` / `qty_multiple`, `trigger` (auto/manual), `route_id`, computed `qty_to_order`, `snoozed_until`, and `supplier_id`. Surfaced in the **Replenishment** view; the **Replenish** button runs the procurement.

Running the rule — `procurement.group.run()` dispatches to each rule's `_run_*`. For Buy, `_run_buy` (a) picks the seller (`supplierinfo_id` → orderpoint `supplier_id` → `product._select_seller`), (b) merges into an existing draft RFQ or creates a new one, (c) adds vendor lead time + company purchase lead time to schedule `date_planned`.

Scheduler — `procurement.group.run_scheduler()` (the "Run Scheduler" cron) evaluates every auto orderpoint and fires Buy/Manufacture procurements — the batch engine turning min/max into draft RFQs.

3.4 Purchase agreements / requisitions

Model `purchase.requisition`. In v19 `requisition_type` is:

- **`blanket_order`** — a fixed-price agreement with one vendor over a period; confirming it updates `product.supplierinfo` so POs to that vendor auto-apply the agreed price. States `draft → confirmed → done` (Closed).
- **`purchase_template`** — a reusable basket of products to spin up POs quickly.

Calls for tender / alternatives — competitive sourcing is done via **PO alternatives** (`alternative_po_ids` + the create-alternative wizard): create competing RFQs to different vendors, compare, confirm the winner, and a warning wizard prompts to cancel the losers.

3.5 Dropshipping

`stock_dropshipping` ships a **Dropship route** + operation type + rule. A product set to Dropship and sold produces, on PO confirmation, a `stock.picking` flagged `is_dropship` whose source is the **vendor location** and destination the **customer location** — goods move vendor → customer with no warehouse stop.

3.6 MPS — Master Production Schedule (Enterprise)

Model `mrp.production.schedule`. A period grid (day/week/month) per product+warehouse with `forecast_target_qty`, min/max replenish, and per-period `forecast_qty`. It rolls starting inventory − forecast − indirect demand + replenish → safety stock, cascades BoM component demand to child schedules, and `action_replenish()` launches procurement (RFQ or MO) ahead of real demand. This is the forecast-driven counterpart to reordering rules.

3.7 Vendor pricelists & lead times

Model `product.supplierinfo`: vendor, `price`, `currency_id`, `min_qty` (price break), validity window (`date_start` / `date_end`), and `delay` (**vendor lead time**, added by `_run_buy` and lead-time calcs). `_select_seller()` picks the best line; confirming a PO writes the actual price back; confirming a blanket order injects agreement pricing here.

3.8 Demo talking points

Set a product's route to Buy + add a supplierinfo with a lead time, create a reordering rule, run the scheduler → a draft RFQ appears with the correct date; walk the RFQ → PO status bar (send, confirm/approve, receipt smart button); validate a partial receipt and show billable-qty control; *(EE)* over-bill vs received to light up the **Release to Pay = Exception** badge, then Force Release; create a **Blanket Order** and show price auto-apply; **Create Alternative** RFQs and confirm a winner; demo **Dropship** end-to-end; *(EE)* enter an MPS forecast and click Replenish; show service **MTO** (SO line spawns a PO line).

4. General Ledger, Accruals & Financial Reporting

4.1 Modules, paths, edition

Module	Path	Edition	Role
`account`	`/odoo/odoo-19-community/addons/account`	**Community**	Core GL: chart of accounts, journals, entries, reconciliation, lock dates, recurring auto-post, the **accrual cut-off wizard**, and the report-engine schema
`account_accountant`	`/odoo/odoo-19-enterprise/account_accountant`	**Enterprise**	Full Accounting app: bank reconciliation, fiscal years, **Deferred Expense/Revenue engine**, lock-date wizard
`account_reports`	`/odoo/odoo-19-enterprise/account_reports`	**Enterprise**	GL / Trial Balance / Balance Sheet / P&L / Aged / Journal reports, **Deferred reports**, tax return / period closing

Community vs Enterprise split (the key point): GL mechanics — accounts, journals, entries, reconciliation, lock dates, recurring entries, the accrual cut-off wizard — are **Community**. **Deferred Expense/Revenue automation** and **all managerial financial reports** are **Enterprise**.

4.2 GL / journal-entry lifecycle

Models `account.move` (header) + `account.move.line` (ledger items).

Chart of accounts — `account.account`; the `account_type` (asset_receivable, asset_cash, asset_current, asset_prepayments, asset_fixed, liability_payable, liability_current, equity, equity_unaffected [Current Year Earnings], income, expense, expense_depreciation, off_balance…) rolls up to `internal_group` (asset/liability/equity/income/expense/off) and drives `include_initial_balance` (whether reports carry a balance across years).

Journals — `account.journal.type`: `sale`, `purchase`, `cash`, `bank`, `credit`, `general` (misc). Deferrals/accruals post to a misc journal.

Entry state machine — `draft → posted → cancel`; independently `payment_state` (not_paid/in_payment/paid/partial/reversed). Posting assigns the legal sequence number, makes lines immutable, and enforces balance + lock dates. Reversal (`_reverse_moves`) is the audit-safe alternative to deletion. Multi-currency lines carry `amount_currency` + `currency_id`.

4.3 Deferred EXPENSE cycle — worked example: prepay 12 months of RENT

Engine: `account_accountant`. **Configure once** (Settings → Accounting): `deferred_expense_account_id` (a Prepayment/`asset_current` account), `deferred_expense_journal_id`, `generate_...method` (`on_validation` auto, or `manual` = "Manually & Grouped"), `deferred_..._computation_method` (`day` / `month` / `full_months`). An identical quartet exists for revenue.

Trigger: on the bill line, set the **Deferred Date** range (e.g. Rent 12,000, 01-Jan → 31-Dec on the Rent expense account).

On posting (`on_validation` mode) Odoo generates:

1. A **reclassification entry** (invoice date): Dr Prepaid Rent 12,000 / Cr Rent Expense 12,000 — parks the whole cost on the balance sheet.
2. **One recognition entry per month** dated month-end: Dr Rent Expense 1,000 / Cr Prepaid Rent 1,000. In `month` mode each month is an equal 1/12; in `day` mode it is calendar-exact; the **last period absorbs rounding** so the slices sum to exactly 12,000.

All generated moves are `auto_post='at_date'` (they sit in draft until the cron reaches their date), and smart buttons link the bill to its deferral schedule. **Net effect:** Prepaid Rent amortises 12,000 → 0 over the year and 1,000 of expense lands in each month's P&L.

Manual/Grouped mode: nothing on posting; the accountant opens the **Deferred Expense Report** and clicks **Generate entry**, which posts one grouped month-end deferral move for all eligible bills and immediately reverses it the next day — the grouped equivalent of the per-bill scheme.

4.4 ACCRUAL cycle — worked example: ELECTRICITY consumed but not billed

Deferrals spread a cost already paid; **accruals** book a cost incurred but not yet billed. Odoo uses the **Automatic Entries ("cut-off") wizard** (`account.automatic.entry.wizard` , Community), with `res.company.expense_accrual_account_id` / `revenue_accrual_account_id` .

Select the consumed-but-unbilled cost, open **Action → Cut-off / Change Period**, pick the accrual date. The wizard books a **balanced pair**:

- **Accrual** at the correct date: Dr Electricity Expense / Cr Expense Accrual (a `liability_current` "accrued expenses" account).
- **Reversal** at the original date: the mirror image.

Because the accrual account is reconcilable, when the real electricity bill finally posts against it, the wizard lines **auto-reconcile** and the accrual washes out — the expense stays in the month it was consumed, and the later bill no longer double-counts. Revenue accrual (delivered/rendered not yet invoiced) is the symmetric case.

4.5 Recurring entries

On `account.move` : `auto_post` = `no` / `at_date` / `monthly` / `quarterly` / `yearly` , with `auto_post_until` and `auto_post_origin_id` . `at_date` = a single future-dated draft the cron posts on its date; `monthly/quarterly/yearly` = a true recurring template that clones itself forward on posting (advancing dates from the origin) until `auto_post_until` . The cron `_autopost_draft_entries` posts due drafts in batches.

4.6 Period close & lock dates

Five independent locks on `res.company` : `fiscalyear_lock_date` (global), `tax_lock_date` (set when a tax closing posts), `sale_lock_date` , `purchase_lock_date` , and `hard_lock_date` (irreversible, no exceptions). Each blocks create/write/unlink on `account.move` at or before its date, enforced in `_post` . Per-user "effective" locks honour temporary `account.lock_exception` grants. Changing a lock goes through the Enterprise `account.change.lock.date` wizard.

Fiscal year — `account.fiscal.year` ranges. Odoo needs no physical P&L closing entry: reports compute **Current Year Earnings** dynamically (P&L accounts reset each year; net income rolls into `equity_unaffected`). Closing a year = post everything → run the tax closing → set `fiscalyear_lock_date` . Tax closing itself is the `account.return` object (see VAT section).

4.7 Financial reports

A two-layer engine: the **schema** is Community (`account.report` , `account.report.line` , `account.report.expression` , `account.report.column` , `account.report.external.value`); the **dynamic rendering + the actual reports** are Enterprise (`account_reports`), one custom handler per report.

Expression engines: `domain` (over move lines), `tax_tags` , `aggregation` (arithmetic over other lines), `account_codes` (prefix match), `external` , `custom` .

Report	Handler
General Ledger	`account.general.ledger.report.handler`
Trial Balance	`account_trial_balance_report`
Balance Sheet / P&L	`balance_sheet` / executive handlers
Executive Summary / Cash Flow	`executive_summary_report` / `account_cash_flow_report`
Aged Receivable / Payable	`account_aged_partner_balance`
Partner Ledger	`account_partner_ledger`
Journal Report	`account_journal_report`
Deferred Expense / Revenue	`account_deferred_reports`
Tax reports / returns	`account_generic_tax_report` / `account_return`

All reports share: date filter + comparison periods, journal/analytic/multi-company filters, drill-down to move lines, and **export to PDF and XLSX**. They read the same posted `account.move.line` rows the deferral/accrual engine produced — so deferred rent shows 12,000 in Prepaid on the Balance Sheet and 1,000/month in the P&L as the entries auto-post.

4.8 Demo talking points

Walk the chart of accounts (`account_type` → reports); create a Misc entry through Draft → Posted, then reverse it; **Deferred Rent** (EE headline): configure, enter a 12,000 bill with a deferred date range, open the Deferred Entries smart button to show the reclassification + 12 monthly entries, open the Deferred Expense Report, then flip to Manually & Grouped; **Accrued Electricity**: use Action → Cut-off, then show auto-reconcile when the real bill lands; set a recurring monthly JE; set a Lock Date and watch a prior edit get blocked; finish on **General Ledger → Trial Balance → Balance Sheet → P&L → Export PDF/XLSX**.

5. VAT / Taxes

> **Version note.** The tax closing / VAT return engine was refactored into a first-class `account.return` object (`account_reports`), replacing the older company-field cron. This is the backbone of the v19 VAT cycle.

5.1 Modules, paths, edition

Module	Path	Edition	Role
`account`	`/odoo/odoo-19-community/addons/account`	**Community**	Taxes, tax groups, fiscal positions, repartition lines, tags (data model)
`account_reports`	`/odoo/odoo-19-enterprise/account_reports`	**Enterprise**	**Tax Report**, `account.return` (VAT return + **tax closing entry**)
`account_reports_cash_basis`	`/odoo/odoo-19-enterprise/account_reports_cash_basis`	**Enterprise**	Cash-basis variant of the tax/GL reports

The tax *data model* is Community; the **Tax Report and VAT return/closing** are Enterprise.

5.2 Tax definition

`account.tax`: `type_tax_use` (sale/purchase/none), `amount_type` (`percent` / `fixed` / `group` / `division` / `code` — the computation method), `amount` , `price_include_override` (per-tax override of the company price-include default), `include_base_amount` / `is_base_affected` (cascading taxes), `tax_group_id` (grouping + carries the closing accounts), `tax_exigibility` (`on_invoice` vs `on_payment` = cash basis), `invoice_repartition_line_ids` / `refund_repartition_line_ids` , `country_id` . `compute_all(...)` computes base + tax amount + the repartition lines to materialise.

Tax repartition lines (`account.tax.repartition.line`) map a tax to **report grids**: `repartition_type` (`base` / `tax`), `factor_percent` (split, e.g. reverse charge +100/−100), `account_id` , `tag_ids` (the +/− grids), and `use_in_tax_closing` (the flag the closing query selects on).

Tax grids (`account.account.tag` , `applicability='taxes'`) are country-scoped; `engine='tax_tags'` report lines sum all move lines carrying a tag. A localization plugs in grids purely by name + country — the l10n chart template creates matching tags and wires them onto each tax's repartition lines.

5.3 Fiscal positions

`account.fiscal.position`: maps source→substitute taxes (`tax_ids`) and income/expense accounts (`account_ids`) by geography, with `auto_apply` , `vat_required` (EU B2B), geographic matching (`country_id` / `country_group_id` / `state_ids` / `zip`), and `foreign_vat` (multi-VAT reporting). It is the single lever for **domestic vs intra-EU vs export vs reverse-charge** — it rewrites the line's taxes before `compute_all` , so the correct grids flow through automatically.

5.4 The VAT cycle

1. **Posting.** For each taxed line, `compute_all` generates the base line (base grids) and a tax line per repartition line, stamped with `tax_line_id` , `tax_repartition_line_id` , `tax_base_amount` , `tax_tag_ids` . Output VAT → tax-payable account; input VAT → tax-receivable. Cash basis holds VAT in `cash_basis_transition_account_id` until payment.
2. **Tax Report** (`account.generic.tax.report.handler`) sums move lines per grid → Net base / Collected (output) / Deductible (input), scoped by country.
3. **VAT return + closing** — `account.return` (states `new → reviewed → submitted → paid → completed`). Periodicity lives on the company (`account_return_periodicity` , reminder day, return journal); returns

auto-generate per period with a deadline + reminder activity.

4. **Tax closing entry** — `action_validate` → `_generate_tax_closing_entries`: a SQL sum over move lines joined to repartition lines filtered on `use_in_tax_closing`, grouped by tax group + account. For each `account.tax.group` it zeroes the period's VAT accounts and moves the net to the group's `tax_payable_account_id` / `tax_receivable_account_id`. The move links back via `closing_return_id` and is posted to the tax-return journal.
5. **Lock + pay authority.** Posting sets `tax_lock_date = date_to`, revokes tax-lock exceptions, and computes the amount to pay; `action_pay` registers the payment to the tax authority. The period is closed and cannot be re-posted.

Cash basis (`account_reports_cash_basis`) recomputes the same report on payment date.

5.5 Demo talking points

Open a 21% sale tax → show `amount_type`, price-include, and the Distribution tabs with the **Tax Grids**; open an "Intra-EU B2B" fiscal position (21%→0% reverse charge, Detect Automatically, VAT required); invoice an EU-VAT partner and point at the tax line's Tags; open **Reporting → Tax Report** for the period; open the period's **VAT Return → Validate** (posts the tax closing entry, sets the tax lock date, shows amount to pay) → **Pay**.

6. Banks

6.1 Modules, paths, edition

Module	Path	Edition	Role
`account`	`/odoo/odoo-19-community/addons/account`	**Community**	Bank journals, statements, payments, reconcile models (data model)
`account_check_printing`	`/odoo/odoo-19-community/addons/account_check_printing`	**Community**	Check payment method + printing
`account_accountant`	`/odoo/odoo-19-enterprise/account_accountant`	**Enterprise**	**Bank reconciliation widget**, reconcile-model auto-matching
`account_reports`	`/odoo/odoo-19-enterprise/account_reports`	**Enterprise**	**Bank Reconciliation Report**
`account_online_synchronization`	`/odoo/odoo-19-enterprise/account_online_synchronization`	**Enterprise**	Bank feeds via OdooFin
`account_batch_payment`	`/odoo/odoo-19-enterprise/account_batch_payment`	**Enterprise**	Grouped in/outbound payments
`account_iso20022`	`/odoo/odoo-19-enterprise/account_iso20022`	**Enterprise**	SEPA Credit Transfer / pain.001 / ISO 20022 XML
`account_bank_statement_import` (+ camt/ofx/qif/csv)	`/odoo/odoo-19-enterprise/…`	**Enterprise**	Statement file import framework + parsers

The bank *data model* is Community; the reconciliation UI, bank feeds, SEPA files, statement import, and batch payments are **Enterprise**. There is no community SEPA/ISO 20022 module in v19.

6.2 Core data model

- **`account.journal`** (`type='bank'`): `bank_account_id` , `bank_statements_source` (undefined/online_sync/imported), `suspense_account_id` (where unreconciled lines rest), `default_account_id` (GL bank account), inbound/outbound payment method lines, and outstanding ("in transit") accounts.
- **`account.bank.statement`** : `balance_start` , computed `balance_end` , `balance_end_real` (bank's stated closing), `line_ids` , and continuous validation (`is_complete` / `is_valid` / `problem_description`).
- **`account.bank.statement.line`** : `_inherits` `account.move` — **each line IS a journal entry** (bank account vs suspense). Fields: `amount` , `payment_ref` / `partner_name` / `account_number` , `is_reconciled` , `running_balance` .
- **`account.payment`** : `payment_type` (inbound/outbound), `partner_type` , `payment_method_line_id` , `state` , `is_matched` (matched to a statement line), `reconciled_invoice_ids` , `destination_account_id` . Methods via `account.payment.method` (built-in `manual`) and method lines.

- **`account.reconcile.model`** — auto-matching rules: `trigger` (manual/auto_reconcile), journal/amount/label/partner match filters, name→partner mapping, and counterpart `line_ids`. (v19 slimmed this model — behaviour is now driven by `trigger` + match filters.)

6.3 Reconciliation widget & Bank Rec Report (Enterprise)

The **reconciliation widget** (`account_accountant`) opens an OWL view over statement lines: a cron runs `auto_reconcile` models in the background; a SQL engine fetches candidate open items per line and a partner-name resolver matches `partner_name` against partners. Reconciling swaps the suspense leg for the matched receivable/payable/other leg and sets `is_reconciled`.

The **Bank Reconciliation Report** (`account_reports`) ties **book** balance to **bank** balance: GL bank balance ± outstanding receipts/payments (items in the journal's outstanding accounts not yet on a statement) = last bank statement closing balance.

6.4 The bank cycle

1. **Create the bank journal**, attach `bank_account_id`, choose the statement source.
2. **Get the statement** — manual entry, **file import** (camt.053/OFX/QIF/CSV parsers → `account.bank.statement` + lines), or **online sync**. Each line posts bank vs suspense; running balance chains and validates against `balance_end_real`.
3. **Reconcile** — per line, Odoo proposes an existing open invoice/payment (by amount + partner + reference), a reconcile-model counterpart (e.g. bank fee → expense + tax), or a new payment; `auto_reconcile` models clear high-confidence lines. Posting clears the suspense leg.
4. **Tie-out** — the Bank Reconciliation Report proves GL + outstanding = bank statement balance.

6.5 Payments, batches, SEPA, checks

- **Register payment** on an invoice → `account.payment` posts to the journal's outstanding account; reconciling the later bank line flips `is_matched` and clears the outstanding account.
- **Batch payments** — `account.batch.payment` groups payments of one method + type with its own state machine.
- **SEPA / ISO 20022** — `account_iso20022` generates **pain.001 SEPA Credit Transfer** (and country variants) XML from a batch payment — the outbound file uploaded to the bank. Enterprise only.
- **Checks** — `account_check_printing` (Community) adds the `check_printing` outbound method + numbering/printing.

6.6 Online bank sync (Enterprise)

`account.online.link` = a connection to an institution via OdooFin (state, `auto_sync`); `account.online.account` = one bank account behind it. Flow: connect institution → discover accounts and link each to a journal → pull transactions, de-duplicate via `online_transaction_identifier`, and **create `account.bank.statement.line` records**; a manual trigger and a cron drive it. From there the normal reconciliation cycle applies.

6.7 Demo talking points

Open the bank journal (statement source, outstanding/suspense accounts) or connect a bank via Online Synchronization; import a CAMT/OFX/CSV statement and show the created statement + running-balance validation; open the **Bank Reconciliation** widget — one line auto-matches an invoice, one matches a reconcile model (bank fee → expense + tax), one creates a payment; on a vendor bill Register Payment → add to a **Batch Payment** → generate the **SEPA pain.001** XML; finish on the **Bank Reconciliation Report** tie-out.

7. Fixed Assets

> **Edition note.** Fixed Assets is **Enterprise-only** — there is no `account_asset` under Community. It `auto_install`s once the Accounting app is present. In v19 the old "deferred revenue" concept is unified into this same model: a deferred/negative asset is just an `account.asset` with a negative original value.

7.1 Modules, paths, edition

Module	Path	Edition	Role
`account_asset`	`/odoo/odoo-19-enterprise/account_asset`	**Enterprise**	Core Assets Management (auto-installs with Accounting)
`account_asset_fleet`	`/odoo/odoo-19-enterprise/account_asset_fleet`	Enterprise	Assets ↔ Fleet bridge
`l10n_in_asset`	`/odoo/odoo-19-enterprise/l10n_in_asset`	Enterprise	India localization
`project_account_asset`	`/odoo/odoo-19-enterprise/project_account_asset`	Enterprise	Assets ↔ project analytics

7.2 Data model

`account.asset` is both the **template** (`state='model'`) and the **live asset** (draft/open/paused/close/cancelled) — same model, discriminated by `state`. The depreciation **board is not a separate line model**; each board line **is a real** **`account.move`** (`asset_id` set, `asset_move_type='depreciation'`), via `depreciation_move_ids`.

Core fields: `method` (`linear` / `degressive` / `degressive_then_linear`), `method_number` (number of depreciations), `method_period` (`'1'`=months / `'12'`=years), `method_progress_factor`, `prorata_computation_type` (`none` / `constant_periods` / `daily_computation`), `original_value`, `salvage_value`, computed `value_residual` and `book_value`, the three accounts (`account_asset_id`, `account_depreciation_id`, `account_depreciation_expense_id`), `journal_id`, `original_move_line_ids` (source bill), `model_id`, and `parent_id` / `children_ids` (revaluation linkage).

7.3 The asset cycle

1. Set up an asset model (`state='model'`) holding method, duration, factor, prorata, salvage %, accounts + journal — no `original_value`. Attach it to a GL account via `account.account.asset_model_ids`.

2. Configure the account to auto-create — on `account.account`: `create_asset` = `no` / `draft` (create in draft) / `validate` (create and validate); `asset_model_ids` (one asset per model); `multiple_assets_per_line` (bill-line quantity spawns that many assets). Only `asset_fixed` / `asset_non_current` accounts qualify.

3. Vendor bill creates/links the asset — posting a bill runs `_auto_create_asset`: for each qualifying line it creates an `account.asset` in draft, pre-filled from the model (`_onchange_model_id`), linked to the bill; if the account is `validate`, the asset is validated immediately. `original_value` is read from the bill line balance (× deductible %, ÷ quantity when multiple). A manual path (`turn_as_asset`) exists from any journal item.

4. Confirm → board generated & posted — `validate()` sets `state='open'`, runs `compute_depreciation_board()`, asserts the last line leaves **0** residual, and posts due entries. The board walks period by period (month-end or fiscal-year-end); each period builds a balanced move **Dr Depreciation Expense / Cr Accumulated Depreciation**. Math: Linear = cumulative-expected minus already-booked (no rounding drift); Degressive = residual × factor per year; Degressive-then-Linear = `max` of the two each period; prorata `none` = full-year convention, else true prorata temporis. Future-dated moves get `auto_post='at_date'` and the standard **auto-post cron** posts each on its date — that is the monthly/yearly automation (no asset-specific cron).

5. Revaluation / modification (`asset.modify` wizard → Re-evaluate) — posts a catch-up depreciation to the operation date, then: **value increase** posts a positive-revaluation move **and creates a child asset** (`parent_id`) depreciating on the same curve (the "Gross Increase" smart button); **value decrease** posts a negative-revaluation move spread over remaining life. Duration and salvage can also change; the board rebuilds forward.

6. Pause / Resume — Pause books the partial period and freezes (`state='paused'`); Resume accumulates the paused interval into `asset_paused_days`, shifting every future period and sliding the end date out.

7. Disposal & sale (`asset.modify` → Sell/Dispose → `set_to_close`) — sets the asset and its children to `close` and posts the disposal entry: reverse gross value, reverse accumulated depreciation, book sale proceeds (if selling), and a **balancing gain/loss** to the company gain/loss account. `net_gain_on_sale` is stored. **Partial disposal** is done by disposing a revaluation child independently. Dispose (scrap) = no proceeds → full loss.

8. Cancellation — `set_to_cancelled()` reverses posted entries, deletes drafts, sets `state='cancelled'`.

State summary: `model → draft → open → (paused ↔ open) / close / cancelled`. Assets with posted entries cannot be deleted; the last board line must leave 0 residual.

7.4 Demo talking points

Open the demo "Asset - 5 Years" and show the board (each row a real journal entry, past posted vs future scheduled); create an asset model; set a fixed-asset account to auto-create + attach the model (+ multiple-assets-per-line); post a vendor bill and show the auto-created asset linked back; Confirm and watch the board post + future entries scheduled; flip method to Declining / Declining-then-Linear and toggle prorata; **Re-evaluate** up (gross-increase child) and down (negative revaluation); **Pause/Resume**; **Sell** with a proceeds line to show the gain/loss entry, contrast with **Dispose**; finish on the Depreciation Schedule report.

8. Manufacturing (MRP)

> **Version note.** Work Orders — the `mrp.workorder` model and its start/pause/finish state machine — ship in **Community** `mrp`. Enterprise `mrp_workorder` ("MRP II") adds the Shop Floor tablet client, Gantt planning, in-work-order quality checks, and HR/IoT bridges, not the model itself.

8.1 Modules, paths, edition

Technical name	Path	Edition	Role
`mrp`	`/odoo/odoo-19-community/addons/mrp`	**Community**	Core MRP: BoM, MO, work centers, **work orders**, unbuild, scrap, Manufacture route
`mrp_account`	`/odoo/odoo-19-community/addons/mrp_account`	**Community**	MO valuation, analytic, cost structure
`mrp_landed_costs`	`/odoo/odoo-19-community/addons/mrp_landed_costs`	**Community**	Landed costs on a MO
`mrp_subcontracting`	`/odoo/odoo-19-community/addons/mrp_subcontracting`	**Community**	Subcontracted manufacturing (BoM type `subcontract`)
`maintenance`	`/odoo/odoo-19-community/addons/maintenance`	**Community**	Equipment & maintenance requests
`mrp_workorder` (MRP II)	`/odoo/odoo-19-enterprise/mrp_workorder`	**Enterprise**	Shop Floor client, WO Gantt planning, quality-in-WO
`mrp_account_enterprise`	`/odoo/odoo-19-enterprise/mrp_account_enterprise`	**Enterprise**	`mrp.report` cost analysis + cost-structure PDF
`mrp_plm` (PLM)	`/odoo/odoo-19-enterprise/mrp_plm`	**Enterprise**	Engineering Change Orders (`mrp.eco`), BoM versioning
`mrp_mps`	`/odoo/odoo-19-enterprise/mrp_mps`	**Enterprise**	Master Production Schedule
`mrp_maintenance`	`/odoo/odoo-19-enterprise/mrp_maintenance`	**Enterprise**	Maintenance tied to work centers
`quality` / `quality_control` / `quality_mrp` / `quality_mrp_workorder`	`/odoo/odoo-19-enterprise/…`	**Enterprise**	Quality points/checks/alerts, in MO and Shop Floor

Community gives a complete manufacturing engine (BoM, MO, work centers, work orders with time logs, unbuild, scrap, Manufacture route, subcontracting, MO valuation, landed costs). **Enterprise** adds Shop Floor/tablet, WO Gantt planning, PLM/ECO, MPS, Quality control, the cost-analysis report, and maintenance↔MRP integration.

8.2 Master data

Bill of Materials — `mrp.bom`: `type` = `normal` ("Manufacture this product") vs `phantom` ("Kit") (+ `subcontract` from `mrp_subcontracting`); `bom_line_ids` (`mrp.bom.line`: component, qty, `operation_id`, `child_bom_id`); `operation_ids` (`mrp.routing.workcenter`); `byproduct_ids` (co-products with `cost_share`); `consumption` (`flexible` / `warning` / `strict` — over/under-consumption tolerance).

Operations — `mrp.routing.workcenter`: label, `workcenter_id`, `time_mode` (auto/manual) + `time_cycle`, `time_total` (setup + cleanup + cycle/efficiency → seeds WO `duration_expected`), and `cost` = time × workcenter cost/hour (the operation cost driver).

Work Center — `mrp.workcenter`**:** `costs_hour`, setup/cleanup time, `time_efficiency`, capacity, `resource_calendar_id`, OEE.

8.3 Manufacturing Order lifecycle — `mrp.production`

`state`**:** `draft` → `confirmed` (rules + component procurement fire) → `progress` (production started) → `to_close` (done, must close) → `done` (closed, moves posted) / `cancel`. Secondary `reservation_state` = MO readiness (Waiting / Ready / Waiting Another Operation); `components_availability_state` (available/expected/late/unavailable).

Flow: **Draft → Confirmed** (`action_confirm` explodes the BoM into `move_raw_ids` + `move_finished_ids`, fires the Manufacture rule) → **reserve** (`action_assign` → Ready) → **progress** → **produce** (set `qty_producing`, `lot_producing_ids` for tracked goods; components consume; over/under checked against BoM `consumption`) → **To Close → Done** (`button_mark_done` validates the finished move, posts moves, recomputes valuation). Under-production offers a **backorder** MO.

8.4 Work orders & Shop Floor — `mrp.workorder`

Model in Community; Shop-Floor UX in Enterprise. `state`**:** `blocked` → `ready` ("To Do") → `progress` → `done` / `cancel`. Fields: `duration_expected` (from routing), `duration` (real, from time logs), `duration_percent`, dates, `workcenter_id`. **Start/Pause/Finish** each write a time log on `mrp.workcenter.productivity` classified by `loss_type` (productive/performance/availability/quality) — feeding OEE and real duration. Enterprise **Shop Floor** (OWL tablet client) lets operators run WOs, register production, answer quality checks; planning is a `web_gantt` view.

8.5 Costing — `mrp_account` (+ enterprise report)

`_cal_price`: `total_cost = Σ component-move value + Σ workorder cost (duration × workcenter cost/hour) + extra_cost × qty`; `finished_move.price_unit = total_cost × (1 − byproduct_cost_share) / qty`. Only recomputed for `fifo` / `average` finished goods. Analytic: work-center time and raw-material moves post analytic lines → per-MO cost picture. Enterprise `mrp.report` (SQL view) gives component vs operation vs total cost, per-unit, and **expected vs actual** — the Cost Analysis pivot + Cost Structure PDF.

8.6 Quality checks (Enterprise)

`quality.point` (the rule: products, operation types, test type, `measure_on` = operation/product/move_line, tolerances) → `quality.check` (`quality_state` none → pass/fail, `measure`, `lot_id`). Test types include measure, passfail, picture, register production/consumed/byproducts, worksheet, spreadsheet. On MO confirmation checks are generated for product/lot/operation; receipts get checks via the picking type; `quality_mrp_workorder` surfaces them in Shop Floor so an operator must pass a check to advance. Non-conformance → `quality.alert`.

8.7 Subcontracting

`mrp_subcontracting`: BoM `type='subcontract'` + `subcontractor_ids` (no operations/by-products). Flow: **buy** the product from a subcontractor → **resupply components** to the subcontractor's location → on **receipt** of the finished good the move is `is_subcontract` and Odoo **auto-creates and records a** `mrp.production` behind the scenes to consume components and value the good; `mrp_subcontracting_account` posts the valuation.

8.8 PLM / Engineering Change Orders (Enterprise)

`mrp.eco`: `type` (bom/product/routing), `bom_id` (current) + `new_bom_id` (draft revision) + `new_bom_revision`, `stage_id`, computed diffs. `state`**:** `new → progress → rebase → conflict → done` (rebase/conflict handle the source BoM changing under the ECO). Stage-gated **approvals** (`mrp.eco.approval.template` / `.approval`); a

stage with `allow_apply_change=True` is where the new revision is applied, bumping `bom.version` and superseding the old BoM.

8.9 MPS, Unbuild, Scrap, Maintenance

- **MPS** (`mrp.production.schedule`) — forecast grid per product+warehouse releasing MOs/POs ahead of demand (see also Procurement §3.6).
- **Unbuild** (`mrp.unbuild`, `draft → done`) — disassembles a finished product from a done MO back into components.
- **Scrap** (`stock.scrap`, `draft → done`) — moves quantity to the scrap location (with `production_id` / `workorder_id` in MRP), removing it from valuation.
- **Maintenance** (`maintenance` + `mrp_maintenance`) — links equipment to work centers; a blocked machine shows on planning; preventive/corrective requests raised from the work center.

8.10 Demo talking points

Build a BoM (Manufacture vs Kit, a component, an operation at a work center; show Flexible Consumption); show the work center cost/hour; create a MO and walk Draft → Confirmed → Check Availability; *(EE)* open **Shop Floor**, start a work order (timer + time log), answer a **Quality Check**, register qty, Finish; mark MO Done and show the **backorder** prompt; *(EE)* open **Cost Analysis** (component vs operation vs total, expected vs actual); add a **landed cost** on the MO; *(EE)* create an **ECO** and walk approval stages → new BoM version; demo **Subcontracting** end-to-end; *(EE)* enter an **MPS** forecast and release MOs; **Unbuild** a done MO and **Scrap** a component.

Appendix — Suggested demo running order

A logical end-to-end story that reuses the same products/partners across modules:

1. **Master data** — create a manufactured product with a BoM, its components, a vendor, and a supplierinfo (price + lead time).
2. **Procurement** — reordering rule → Run Scheduler → RFQ → confirm → receipt → vendor bill → *(EE)* 3-way match → payment.
3. **Inventory & cost** — show the receipt's valuation entry; add a **landed cost**; adjust a quant.
4. **Manufacturing** — confirm a MO for the finished product, run the work order on **Shop Floor** with a quality check, close it, open **Cost Analysis**.
5. **Sales/delivery** (brief) — deliver the finished good; show the COGS entry.
6. **General Ledger** — book a **deferred rent** bill and an **accrued electricity** cut-off; show them on the P&L/Balance Sheet.
7. **Assets** — post a vendor bill that **auto-creates a fixed asset**; confirm it; show the depreciation board.
8. **VAT** — run the **Tax Report** and post the **VAT return / tax closing**.
9. **Banks** — import/sync a statement, **reconcile**, and show the **Bank Reconciliation Report** tie-out.
10. **Reporting & close** — General Ledger, Trial Balance, Balance Sheet, P&L, export to PDF; set a **lock date**.

Every step above is backed by the modules and cycles documented in this study, all present and functional in Odoo 19 Enterprise.